

实现流程

1. 定义联系人的数据结构（结构体）
2. 创建 存储联系人数据的 static 全局数组 和 记录联系人实际数量的 static 全局变量。
3. 搭建电话簿的主业务逻辑框架（工作流程）
4. 定义并实现相应的功能函数
 1. 添加联系人
 2. 删除联系人
 3. 查找联系人
 4. 显示联系人
5. 组织上述功能函数的调用逻辑。
6. 整体测试、微调。

框架搭建1

main.c

```
1 // 管理程序运行的主业务逻辑
2 #define _CRT_SECURE_NO_WARNINGS
3 #include "telephoneBook.h"
4
5 // 定义枚举，使用枚举常量描述事件
6 enum {ADD=1, DEL, SREACH, SHOW, EXIT};
7
8 // 显示菜单
9 void show_menu(void)
10 {
11     printf("-----显示菜单-----\n");
12 }
13
14 // 电话簿启动、工作
15 void startWorking(void)
16 {
17     // 电话簿具体工作内容
18     while (1)
19     {
20         // 显示菜单
21         show_menu();
22
23         // 接收用户输入(1.添加, 2.删除, 3.查找, 4.显示, 5.退出)
24         int number = 0;
25         printf("请在菜单中选择您的需求: ");
26         (void)scanf("%d", &number);
27
28         // 根据用户输入，作用相应操作
29         switch (number)
30         {
31             case ADD:
```

```

32         printf("1. 添加联系人\n");
33         break;
34     case DEL:
35         printf("2. 删除联系人\n");
36         break;
37     case SREACH:
38         printf("3. 查找联系人\n");
39         break;
40     case SHOW:
41         printf("4. 显示联系人\n");
42         break;
43     case EXIT:
44         printf("5. 退出系统\n");
45         return;        // 结束整个电话簿系统
46     default:
47         break;
48     }
49 }
50 }
51
52 int main(void)
53 {
54     startWorking();
55
56     system("pause");
57     return EXIT_SUCCESS;
58 }

```

telephoneBook.c

```

1  // 管理电话簿 相关信息
2  #include "telephoneBook.h"
3
4  // 定义 结构体数组 存储联系人
5  static person_t numbers[MAX_PERSON_NUM];
6
7  // 描述数组中，实际存储联系人个数
8  static int size = 0;
9
10 // 具体添加联系人
11
12 // 具体删除联系人
13
14 // 具体查找联系人
15
16 // 显示所有联系人

```

telephoneBook.h

```

1  // #pragma once    // windows 系统自带防止头文件重复包含。
2
3  #ifndef _TELEPHONE_BOOK_H_
4  #define _TELEPHONE_BOOK_H_

```

```

5
6 // 系统库头文件引入
7 #include <stdio.h>
8 #include <string.h>
9 #include <stdlib.h>
10 #include <math.h>
11 #include <time.h>
12
13 // 函数声明
14 void show_menu(void);
15
16 // 宏定义，常量描述数组容量
17 #define MAX_PERSON_NUM 2048
18
19 // 定义联系人结构体
20 typedef struct person {
21     char name[32]; // 联系人姓名
22     char phone[16]; // 联系人电话
23 } person_t;
24
25
26 #endif

```

框架搭建2

main.c

```

1 // 管理程序运行的主业务逻辑
2 #define _CRT_SECURE_NO_WARNINGS
3
4 #include "telephoneBook.h"
5
6 // 定义枚举，使用枚举常量描述事件
7 enum {ADD=1, DEL, SREACH, SHOW, EXIT};
8
9 // 显示菜单
10 void show_menu(void)
11 {
12     printf("\n欢迎使用简易电话簿系统\n\n");
13     printf("\t1. 添加联系人\n\n");
14     printf("\t2. 删除联系人\n\n");
15     printf("\t3. 查找联系人\n\n");
16     printf("\t4. 显示联系人\n\n");
17     printf("\t5. 退出系统\n\n");
18 }
19
20 // ADD分支
21 void add(void)
22 {
23     printf("\n\t【姓名】为: ");
24     char name[32] = { 0 }; // 临时获取用户输入，存储联系人姓名
25     (void)scanf("%s", name);
26
27     printf("\n\t【电话】为: ");

```

```

28     char phone[16] = { 0 };          // 临时获取用户输入，存储联系人姓名
29     (void)scanf("%s", phone);
30
31     // 向联系人数组中，添加一个新联系人
32     add_person(name, phone);
33
34     printf("\n联系人【%s】添加成功!\n", name);
35 }
36
37 // DEL分支
38 void del(void)
39 {
40     printf("\n\t待【删除】的联系人姓名为: ");
41     char name[32] = { 0 };          // 临时获取用户输入，存储联系人姓名
42     (void)scanf("%s", name);
43
44     // 从联系人数组中，删除联系人
45     del_person(name);
46
47     printf("\n联系人【%s】已成功删除!\n", name);
48 }
49
50 // SEARCH分支
51 void search(void)
52 {
53     printf("\n\t待【查找】的联系人姓名为: ");
54     char name[32] = { 0 };          // 临时获取用户输入，存储联系人姓名
55     (void)scanf("%s", name);
56
57     // 从联系人数组中，查找联系人的电话
58     char *phone = search_person(name);
59     if (NULL == phone)
60     {
61         printf("\n抱歉! 电话簿中【%s】查无此人!\n", name);
62     }
63     else
64     {
65         printf("\n联系人【%s】的电话为: %s\n", name, phone);
66     }
67 }
68
69 // SHOW分支 --- 可以不封装该函数。 直接调用 telephoneBook.c 中的 show_person()
70 //void show(void)
71 //{
72 //    printf("4. 显示联系人\n");
73 //}
74
75 // 电话簿启动、工作
76 void startWorking(void)
77 {
78     // 电话簿具体工作内容
79     while (1)
80     {
81         // 显示菜单
82         show_menu();
83
84         // 接收用户输入(1.添加, 2.删除, 3.查找, 4.显示, 5.退出)
85         int number = 0;

```

```

86     printf("请在菜单中选择您的需求: ");
87     (void)scanf("%d", &number);
88
89     // 根据用户输入, 作用相应操作
90     switch (number)
91     {
92     case ADD:
93         add();
94         break;
95     case DEL:
96         del();
97         break;
98     case SREACH:
99         search();
100        break;
101    case SHOW:
102        show_person();
103        break;
104    case EXIT:
105        printf("\n欢迎下次再使用电话簿系统, 再见~~\n");
106        return;    // 结束整个电话簿系统
107    default:
108        break;
109    }
110 }
111 }
112
113 int main(void)
114 {
115     startWorking();
116
117     system("pause");
118     return EXIT_SUCCESS;
119 }
120

```

telephoneBook.c

```

1  // 管理电话簿 相关信息
2  #include "telephoneBook.h"
3
4  // 定义 结构体数组 存储联系人
5  static person_t numbers[MAX_PERSON_NUM];
6
7  // 描述数组中, 实际存储联系人个数
8  static int size = 0;
9
10 // 具体添加联系人
11 void add_person(const char* name, const char* phone)
12 {
13 }
14
15 // 具体删除联系人
16 void del_person(const char* name)
17 {
18 }
19

```

```

20
21 // 具体查找联系人 ---- 返回联系人 的电话
22 char *search_person(const char* name)
23 {
24     return NULL;
25 }
26
27 // 显示所有联系人
28 void show_person(void)
29 {
30
31 }

```

telephoneBook.h

```

1 // #pragma once // windows 系统自带防止头文件重复包含。
2
3 #ifndef _TELEPHONE_BOOK_H_
4 #define _TELEPHONE_BOOK_H_
5
6 // 系统库头文件引入
7 #include <stdio.h>
8 #include <string.h>
9 #include <stdlib.h>
10 #include <math.h>
11 #include <time.h>
12
13 // 函数声明
14 void add_person(const char* name, const char* phone);
15 void del_person(const char* name);
16 char* search_person(const char* name);
17 void show_person(void);
18
19 void show_menu(void);
20 void add(void);
21 void del(void);
22 void search(void);
23
24 // 宏定义, 常量描述数组容量
25 #define MAX_PERSON_NUM 2048
26
27 // 定义联系人结构体
28 typedef struct person {
29     char name[32]; // 联系人姓名
30     char phone[16]; // 联系人电话
31 } person_t;
32
33 #endif

```

具体功能函数实现

添加联系人

```
1 // 具体添加联系人
2 void add_person(const char* name, const char* phone)
3 {
4     // 校验参数
5     if (NULL == name || NULL == phone)
6     {
7         printf("add_person 添加联系人失败! 参数错误! \n");
8         return;
9     }
10
11     strcpy(numbers[size].name, name); // 存姓名到数组
12     strcpy(numbers[size].phone, phone); // 存电话到数组
13
14     ++size; // 更新size
15 }
```